Sistema de Gestão de Rebanho: Justificativa Teórica da Modelagem

Andrei Ruan Steffens, Estela Giovana Kaster, Paulo Henrique Vieira Klumb, Matheus Zuchi Balbinot

[1]Instituto Federal de Educação, Ciência e Tecnologia Catarinense
Campus Concórdia – Concórdia, SC – Brasil

andreiruansteffens@gmail.com, estelagkaster@gmail.com,

pauloh152008@gmail.com, mdscriaconta@gmail.com

1. Introdução

O sistema serve para controlar o rebanho de uma propriedade rural. Ficamos só com gado de corte. Com ele o proprietário cadastra os animais, acompanha peso, vacinas e alimentação, compra insumos e vende os animais para o frigorífico.

2. Casos de Uso

O diagrama tem quatro atores. O Proprietário é quem mais usa o sistema: cadastra animais, monitora crescimento e peso, gerencia a nutrição do gado, registra vacinação, realiza inseminação e compra insumos. O Frigorífico consulta os animais disponíveis para compra, compra os animais e vê o histórico de compras. O Fornecedor de Insumos cadastra e vende insumos. O Usuário faz o autocadastro.

O cadastro aparece em Cadastrar Animal e Cadastrar Insumos, e a consulta em Consultar Animais Disponíveis, Consultar Alimentação do Animal e Histórico de Compras.

Usamos *include* quando um caso de uso sempre precisa de outro para acontecer [1]. Realizar Inseminação inclui Consultar Genética, porque antes de inseminar é preciso ver a genealogia da fêmea. Gerenciar Nutrição do Gado inclui Consultar Alimentação do Animal. E Identificar Animais para Venda tem um *include* com Vender/Comprar Animal, já que a identificação é feita para chegar na venda. Não encontramos nenhum caso opcional que pedisse *extend*.

3. Diagrama de Classes

São onze classes: `Animal`, `Femea`, `Macho`, `Pesagem`, `Vacinacao`, `Proprietario`, `BaseAlimentacao`, `Insumo`, `Fornecedor`, `Frigorifico` e `CompraAnimal`.

3.1. Abstração e Herança

`Animal` é abstrata, porque no sistema todo animal é fêmea ou macho e não faz sentido criar um animal sem saber qual dos dois ele é. Ela guarda o que é comum, como brinco, data de nascimento, raça, pesagens e vacinas, e deixa `categoria()` e `peso_minimo_abate()` como métodos abstratos. `Femea` e `Macho` herdam de `Animal`. Separamos por sexo porque o manejo muda: só a fêmea tem

prenhe, inseminar() e registrar_parto(), e só o macho tem castrado e castrar(). Desse jeito não tem como chamar inseminar() em um macho, porque o método não existe na classe dele.

3.2. Encapsulamento

Todos os atributos são privados (−) e só mudam pelos métodos da própria classe [2]. Por exemplo, vendido só muda por marcar_vendido(), e prenhe só muda por inseminar() ou registrar_parto(). O estoque de um insumo também não é alterado direto, só por baixar_estoque(). Assim a validação fica em um lugar só e o resto do sistema não consegue deixar um objeto em estado errado.

Daqui saem as duas regras de negócio do sistema. A primeira é que um animal só pode ser vendido ao frigorífico quando pode_vender() permite, e depois da venda ele é marcado como vendido para não ser vendido de novo. A segunda é que só fêmeas podem ser inseminadas, e antes disso pode_inseminar() verifica se a inseminação com aquele touro é possível.

3.3. Polimorfismo

Femea e Macho sobrescrevem categoria() e peso_minimo_abate(), então cada uma devolve a sua categoria e o seu peso mínimo para abate. Como pode_vender() está em Animal, ela pode chamar peso_minimo_abate() sem precisar saber se o objeto é fêmea ou macho.

3.4. Agregação e Composição

Escolhemos pelo ciclo de vida das partes. Pesagem e Vacinacao são composição de Animal, porque uma pesagem ou uma vacina registrada não existe sem o animal. CompraAnimal é composição de Frigorifico, pois o registro faz parte do histórico daquele frigorífico.

O Proprietario agrega os animais e as bases de alimentação, e o Fornecedor agrega os insumos. Aqui a parte continua existindo sem o todo. O resto são associações: o proprietário compra insumos, a base de alimentação usa um insumo, cada animal tem no máximo uma base de alimentação, e cada CompraAnimal se refere a um animal. Também tem a associação entre Femea e Animal, com os papéis de mãe e filhos, que é o que permite consultar a genética.

4. Considerações Finais

Deixamos as regras dentro das classes para que a interface em Tkinter só chame os métodos, como pede o trabalho. Isso também deve facilitar os testes na segunda parte.

Referências

[1] VISUAL PARADIGM. *Include and Extend Use Case Diagram*. Disponível em: https://online.visual-paradigm.com/pt/diagrams/templates/use-case-diagram/include-and-extend-use-cases. Acesso em: 30 out. 2026.

[2] LUCID SOFTWARE. *Tutorial de diagrama de classes UML*. Disponível em: https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-classes. Acesso em: 30 out. 2026.